

Core Java Concepts

- Object-Oriented Programming (OOP) is the foundation of Java. You'll need to be well-versed in the four main principles:
- **Encapsulation:** Wrapping data (variables) and code (methods) together as a single unit. Using access modifiers (private, public, protected) ensures controlled access to variables.
- **Inheritance:** Allows a new class to inherit the properties and behaviors of an existing class, promoting code reuse.
- **Polymorphism:** The ability of one object to take many forms, allowing methods to act differently based on the object that calls them.
- **Abstraction:** Hiding the internal details of implementation and exposing only necessary information to the user.

```
class Animal {  
    void sound() {  
        System.out.println("Generic Animal Sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Animal myDog = new Dog();  
        myDog.sound(); // Output: Bark  
    }  
}
```

In this example, polymorphism is illustrated by overriding the `sound()` method in the `Dog` class.

Java Collections Framework

The Java Collections Framework is a powerful set of classes and interfaces for storing and manipulating data. It includes:

- **List (e.g., ArrayList, LinkedList):** Ordered collection that allows duplicates.
- **Set (e.g., HashSet, TreeSet):** Unordered collection that does not allow duplicates.
- **Map (e.g., HashMap, TreeMap):** Collection of key-value pairs.
- **Queue (e.g., LinkedList, PriorityQueue):** Collection that allows elements to be processed in a specific order.

```
import java.util.HashMap;

public class Example {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new
HashMap<>();
        map.put("Java", 1);
        map.put("Python", 2);
        System.out.println(map.get("Java")); // Output:
1
    }
}
```

Here, we use a HashMap to store and retrieve values based on keys.

Exception Handling

Exception handling is key to writing robust and fault-tolerant Java programs. Java provides several constructs to handle runtime and compile-time errors:

- **Try-Catch:** The try block contains code that might throw an exception. The catch block handles the exception if it occurs.
- **Throws vs. Throw:** The throws keyword declares that a method might throw an exception, while throw is used to explicitly throw an exception.
- **Checked vs. Unchecked Exceptions:** Checked exceptions are exceptions that need to be explicitly handled, while unchecked exceptions (e.g., `ArithmeticException`) do not.

```
public class Example {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0; // ArithmeticException  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero.");  
        }  
    }  
}
```

The code above catches the `ArithmeticException` caused by dividing by zero, preventing the program from crashing.

Java 8 Features

Java 8 introduced several key features that have become industry standards:

Lambda Expressions

Lambda expressions enable you to pass behavior as parameters. They provide a concise way to express instances of single-method interfaces (functional interfaces).

```
List<String> names = Arrays.asList("John", "Jane",  
"Alex");  
names.forEach(name -> System.out.println(name));
```

Streams API

The Streams API enables you to process sequences of elements in a functional style. It allows you to perform operations such as filtering, mapping, and reducing in a more readable way.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);  
    numbers.stream()  
        .filter(n -> n % 2 == 0)  
        .forEach(System.out::println); // Output: 2, 4
```

Optional

Optional is a container object that may or may not contain a value, helping to avoid `NullPointerException`.

```
Optional<String> str = Optional.ofNullable("Hello");  
System.out.println(str.orElse("Default")); // Output:  
Hello
```

Java 11 Features

Java 11 was a major LTS (Long-Term Support) release and included several important updates:

New String Methods

Java 11 introduced several new methods for the String class:

- **isBlank()**: Checks if a string is empty or contains only whitespace.
- **lines()**: Splits the string into multiple lines.
- **strip()**: Removes leading and trailing whitespace.

```
String text = " Hello, World! ";
```

```
System.out.println(text.strip()); // Output: "Hello, World!"
```

Local-Variable Syntax for Lambda Parameters

Java 11 allows the use of `var` in lambda expressions for greater clarity and conciseness.

```
List<String> list = Arrays.asList("apple", "banana",  
                                "cherry");  
list.forEach((var fruit) -> System.out.println(fruit));
```

HTTP Client API

Java 11 added a modern HTTP Client API to handle HTTP requests more effectively, replacing the older `URLConnection` class.

Java 17 Features

Java 17 is another LTS version, and it brings several new features that significantly improve performance and simplify coding.

Sealed Classes

Sealed classes allow developers to define a limited set of subclasses, improving code maintainability and safety.

```
public abstract sealed class Animal permits Dog, Cat { }  
public final class Dog extends Animal { }  
public final class Cat extends Animal { }
```

Pattern Matching for instanceof

Java 17 simplifies instanceof checks by adding pattern matching, removing the need for explicit casting.

```
Object obj = "Hello, Java!";  
if (obj instanceof String s) {  
    System.out.println(s.toUpperCase()); //  
Output: HELLO, JAVA!  
}
```

JEP 356: Enhanced Pseudo-Random Number Generators

Java 17 improves the way random numbers are generated with enhanced algorithms, making it more secure and faster.

Multithreading & Concurrency

Java's multithreading and concurrency libraries are powerful tools for building high-performance applications. Key topics include:

- **Thread Creation:** Understanding how to create and manage threads using `Thread` and `Runnable`.
- **Executor Service:** A higher-level framework for managing a pool of threads, allowing for more flexible and efficient thread management.
- **Synchronization:** Ensuring that only one thread can access a block of code at a time to prevent data corruption.
- **Fork-Join Framework:** Dividing tasks into smaller subtasks that can be executed in parallel, ideal for CPU-intensive operations.

Design Patterns in Java

Familiarity with common design patterns is crucial for writing scalable, maintainable code:

- **Singleton:** Ensures a class has only one instance.
- **Factory:** Allows subclasses to change the type of objects created.
- **Observer:** Notifies multiple objects about changes to a subject.

Conclusion

Mastering Java requires understanding core concepts, being well-versed in its evolving features (especially Java 8, 11, and 17), and practicing through real-world examples. Familiarity with the Java Collections Framework, exception handling, and concurrency will be essential in almost every interview.

Keep revisiting these topics, and remember: consistency and practice are key to mastering Java.